

Sistemas Inteligentes

Dr. Marco Aurelio Nuño Maganda

Universidad Politecnica de Victoria
Ingenieria en Tecnologias de la Informacion
Cuatrimestre Enero - Abril 2025

mnunom@upv.edu.mx

January 6, 2026

Breve CV del Facilitador

- Doctor en Ciencias Computacionales por parte del INAOE (2009).
- Profesor de Tiempo Completo de la UPV desde 2009.
- Miembro del Sistema Nacional de Investigadores - Nivel Candidado (2014-2016), Nivel I (2020-2022), Nivel I (2023-2027)
- 22 tesis dirigidas a nivel maestría.
- Asignaturas impartidas en el pasado
 - Licenciatura: Cómputo en Dispositivos Móviles, Graficación por Computadora Avanzada, Lenguajes y Automatas, Programación Orientada a Objetos
 - Maestría: Visión por computadora, Tópicos Selectos de Imagenología, Fundamentos de Sistemas de Información
- Miembro del Núcleo Académico Básico (NAB) tanto de la Maestría en Ingeniería (MI) / Doctorado en Ciencia y Tecnología Aplicada (DCYTA) de la UPV.

Horario de la Clase

■ Días y horas de clase

	Lunes	Martes	Miércoles	Jueves	Viernes
ITI-271362	13:00-13:55	13:00-13:55	13:00-13:55	11:10-12:05	11:10-13:00
ITI-271369	11:10-13:00	11:10-12:05	11:10-12:05	13:00-13:55	08:50-9:45

■ Fechas Importantes:

- Inicio de Cursos: 8/ENERO
- Fin de Cursos: 25/Abril (16/Abril)
- Dias no hábiles oficiales: 3/Febrero (Lunes), 17/Marzo (Lunes), 17 y 18 de Abril (Jueves/Viernes).

Plataforma Virtual para el Curso

- Nombre de la clase: **Sistemas Inteligentes- Enero - Abril 2025**
- Código de clase en Classroom: **fwh6pnc**
- Enlace Meet para sesiones no presenciales:
<https://meet.google.com/bsd-wqyz-fhx>

Reglas básicas

- Se recomienda puntualidad y asistencia a las sesiones.
- Respeto hacia el profesor y hacia sus compañeros y compañeras.
- No se permite el ingreso y/o ingestión de **Alimentos** ni **Bebidas** de ningún tipo a la clase.
- No se permite usar **AUDIFONOS O DISPOSITIVOS MANO-LIBRES EN CLASE**. De detectar esta situación, se asignará al estudiante **un proyecto EXTRA**, y de reiterar dicha situación, se le asignará DOBLE, TRIPLE, CUADRÚPLE **proyecto extra** sin derecho a réplica.
- En caso de que algún estudiante sea sorprendido realizando actividades ajenas a la clase (por ejemplo, Jugando ClashRoyale), se aplicará la misma penalización del punto anterior.

Uso del Teléfono Inteligente

Resguardo del teléfono inteligente

- En caso de que la clase no lo requiera, se solicitará al INICIO de la CLASE a todos los asistentes a la clase (incluyendo al profesor) guardar su teléfono en una caja, la cual será cerrada, regresando su teléfono al finalizar la SESION.
- Los telefonos deben configurarse en modo Silencio, ya que en caso de que suene estando bajo resguardo, dicho celular quedará a resguardo del profesor hasta la proxima clase.



Ausencia de Grupo Completo

- Se recomienda evitar conspiraciones para obtener clases libres aún cuando tengan el resto del día libre.
- Lo anterior puede hacer enojar a las autoridades, por lo que además de hacer algún coraje, puede afectarles en su salud física y mental.
- Si por algún motivo no tiene clase previamente, les invito a manifestar su inconformidad por escrito o por correo electrónico.

Penalización

En caso de que un grupo se encuentre ausente al inicio de una sesión, dicho grupo se hace acreedor a trabajo extra OBLIGATORIO, el cual será definido en el transcurso de dicha sesión y hecho de su conocimiento mediante la plataforma de LMS utilizada para el curso.

Para poder registrar calificación aprobatoria, el estudiante debe cumplir con lo siguiente:

- Tener calificación aprobatoria en todas las unidades (100-100-40 no da calificación aprobatoria).
- Tener por lo menos dos asesorías por semana (Registrarlas por semana, no 30 asesorías al final del cuatrimestre)
- Cumplir con el 80% de asistencia mínimo, incluyendo aquellas inasistencias justificadas debidamente mediante el SIITA

Pase de Lista

- Se pasa lista al inicio de la clase. En caso de reincorporación tardía, se pone un retardo.
- DOS RETARDOS equivalen a una INASISTENCIA, que no es JUSTIFICABLE.
- Para justificar una inasistencia, es necesario cumplir con los siguientes pasos:
 - Subir al apartado de classroom *Justificación de Inasistencias* un PDF la evidencia de justificación con el siguiente formato:
iti-271154_APAT_AMAT_NOM_DD_MM_YY.pdf
 - APAT es apellido paterno, AMAT es apellido materno y NOMBRES son nombres, separados por guiones bajos.
 - DD,MM y YY es día, mes y año a dos dígitos de su inasistencia.
 - Si la inasistencia fue por varios días, duplicar la entrega tantos días como haya faltado, pero solo los días en los que corresponde a la clases y siempre y cuando lo anterior se haga antes de los 7 días posteriores a la inasistencia.
 - Solo se justifican inasistencias por motivos médicos incapacitantes. Cualquier otra causa no es justificable.

Alumnos con Empleo (1)

Alumnos VIPs

- En caso de tener un empleo formal dentro o fuera de la ciudad, es necesario entregar una **constancia laboral** que acredite el horario que se esta cubriendo (en el caso de locales, este horario se debe empalmar con el de la materia). En esa constancia debe acreditar que se esta haciendo labores de manera presencial en tal ubicacion. Esto lo dispensa solo del requisito de las asistencias, mas no de los proyectos que deban entregarse.
- Se podrá solicitar la presentación de avance/proyecto de manera “remota” durante alguna de las clases, y de **NO CUMPLIR** con lo anterior, dicho proyecto será **DESCARTADO**.
- Enviar esa constancia con copia para el director de carrera.

Alumnos con Empleo (2)

- La justificación de inasistencias por *actividad laboral* se considerará a partir del momento de la recepción de dicha constancia en el correo del instructor (y no a partir de la fecha indicada en la constancia), por lo que si se recibe de manera tardía (con mas de una semana de retardo), dichas inasistencias NO SERAN justificadas.
- La justificación será válida si el estudiante programa **POR LO MENOS** dos asesorías por semana. De no hacerlo, pierde el beneficio de la justificación y se aplican las reglas anteriormente establecidas.

- 1 Introducción a la Inteligencia Artificial
 - 1 Inteligencia Artificial y Sistemas Inteligentes
 - 2 Razonamiento y representación del conocimiento
 - 3 Estrategias de Búsqueda
- 2 Técnicas Básicas de Aprendizaje
 - 1 Aprendizaje Supervisado y No Supervisado
 - 2 Aprendizaje basado en Redes Neuronales
 - 3 Algoritmos Genéticos
- 3 Introducción a la percepción automática
 - 1 Percepción Visual
 - 2 Percepción Visual Estereoscópica
 - 3 Procesamiento de Lenguaje Natural

Evaluación (1)

- Para cada unidad del curso, se consideran 3 aspectos:
 - Ejercicios o investigaciones especiales (1)- 25%
 - Proyecto Individual - 35%
 - Proyecto en Equipo - 40%
- Para aprobar el curso, es obligatorio:
 - Tener calificación aprobatoria en todas las unidades (100-100-40 no da calificación aprobatoria).
 - Tener por lo menos dos asesorías por semana (Registrarlas por semana, no 30 asesorías al final del cuatrimestre)
 - Cumplir con el 80% de asistencia mínimo, incluyendo aquellas inasistencias justificadas debidamente mediante el SIITA

Evaluación (3)

Tipos de sesiones a lo largo del curso:

- Sesiones de teoría/práctica. Se presentan varios temas y se explican códigos y demos preparados con anterioridad
- Sesiones de seguimiento de proyectos. Se revisan los avances de proyectos individuales o en equipo.
- Sesiones de esparcimiento. Se permitirá a los estudiantes trabajar en proyectos pendientes, pero deben estar presentes en clase, además de contabilizar asistencia.

Evaluación (3)

Acerca de los proyectos:

- Aleatorios y DIFERENTES para la mayoría (preferentemente para cada integrante)
- Equipos: Proyectos diferentes para cada equipo, e Integrantes de los mismos formados de manera ALEATORIA!!

Fragmentación de equipos

Penalización

- Si llegar a ocurrir que en un proyecto en equipo no hay un acuerdo para trabajar en equipo (Hay dos o mas entregas del proyecto asignado por partes diferentes dentro del mismo equipo), cada “fragmento” de equipo recibe una penalizacion de **25 puntos** mas las penalizaciones acumuladas por otros rubros.
- Si un “fragmento” de proyecto es entregado por un solo individuo, a partir de ese momento se considera que tal individuo tiene capacidad para elaborar proyecto complejos en INDIVIDUAL, por lo que para futuros proyectos **NO SE LE CONSIDERARÁ** para formar equipos.
- Esta regla **NO APLICA** cuando hay uno o varios “desertores” del equipo (y hay una sola entrega del proyectos en equipo)

Acerca de Exención

- Cuando el profesor realizar alguna mecánica para excentar un proyecto (Individual/Equipo/Asignación especial) y uno o varios estudiantes completan lo solicitado, existen dos posibilidades:
 - El estudiante acepta excentar la elaboración de dicho proyecto o actividad, pero al hacer esto asume que la calificación asignada es 70.
 - El estudiante decide hacer el proyecto a pesar de haber excentado. En este caso el estudiante se hace acreedor a 20 puntos que puede aplicar sobre la calificación de dicho proyecto.

Cartucho de Recuperación (REC) (1)

- Estudiante tiene derecho a solicitar SOLAMENTE UNA VEZ un proyecto de recuperación aplicable a un solo proyecto o actividad durante todo el cuatrimestre.
- Esta solicitud debe HACERLA es estudiante - El profesor NO ES RESPONSABLE de informar al estudiante cuando tiene un ADEUDO. Cuando el estudiante realice dicho acercamiento, es responsabilidad que haya un respaldo del proyecto de recuperación, incluyendo fechas de entrega.
- A partir de la primera solicitud de recuperación de un estudiante, se asigna una fecha de entrega. Este tiempo corre para todos aquellos estudiantes que tengan adeudo, por lo que si otros estudiantes se acercan de manerá tardía, tendrán el mismo tiempo para la entrega del proyecto.
- Si alguien hace la solicitud de recuperación posterior a la fecha de entrega asignada inicialmente, el proyecto **YA NO SE PUEDE RECUPERAR.**

Cartucho de Recuperación (REC) (2)

- Si el proyecto no entregado es individual, se asigna otro proyecto diferente.
- Si el proyecto es en equipo, de común acuerdo con los integrantes pueden trabajar en otro proyecto diferente en equipo, o recibir una asignación individual de un proyecto diferente.
- El proyecto de recuperación será asignado siempre y cuando el estudiante cumpla lo siguiente:
 - El estudiante debe tener el porcentaje de asistencia mínimo necesario para aprobar.
 - El estudiante ha agendado el porcentaje de asesorías en el SIITA.
- El nuevo proyecto asignado está diseñado para que el estudiante invierta en él por lo menos 2 SEMANA. Si lo solicita un día antes de terminar el cuatrimestre, posiblemente no tendrá tiempo de llevarlo a cabo.

Reporte Técnico de Desarrollo de Práctica

- Para cada práctica realizada, entregar un documento (**únicamente en formato PDF***) con las siguientes secciones:
 - Introducción
 - Desarrollo Experimental
 - Resultados
 - Conclusiones
 - **Referencias**
- Para GENERAR este reporte es necesario utilizar la plantilla en LATEX (**únicamente usando LATEX***) localizada en el siguiente enlace:
<https://www.overleaf.com/read/dgkhvfwynygvc>

Reporte Técnico de Desarrollo de Práctica

- Bajo ninguna circunstancia deben incluir **CÓDIGO FUENTE**. Si pueden incluir diagrama de flujo, Pseudocódigo, Diagrama E-R, Diagrama de Clases, de Casos de USO, etc. De incluir código fuente, solo tendrá un 50% del valor en la calificación.
- En caso de trabajos individuales o en EQUIPO, deben emplear la plantilla LaTeX que se provee. En caso de utilizar algo diferente a LaTeX u otra plantilla de LaTeX, la calificación proporcional del informe será **DESESTIMADA**.
- En caso de trabajos en equipo, se debe agregar los integrantes al inicio del INFORME. **El trabajo solo cuenta para aquellos integrantes mencionados en el informe (y que dicho nombre se encuentre registrado tal cual en la lista). Una vez ENTREGADO, si hay OMISIONES de los integrantes, no se realizará CORRECCION alguna, se debe asumir la consecuencias que esto conlleva.**

Ponderación del Informe en la Calificación del Proyecto

- Informe: 34 Puntos
 - Uso adecuado de Latex: 5 Puntos
 - Organización y Redacción: 6 Puntos
 - Referencias en formato adecuado: 8 Puntos
 - Evidencia del trabajo realizado: 8 Puntos
 - Sin faltas de ortografía ni errores de dedo: 7 Puntos
- Proyecto: 66 Puntos
 - Ejecución y Funcionalidad: 45 Puntos
 - Modularidad: 13 Puntos
 - Documentación: 8 Puntos

Entregables de proyecto individual (1)

- Crear un archivo ZIP con el siguiente formato de nombre:
 - **iti-271154_uX_nuno_maganda_marco_aurelio**
- Dentro, debe contener lo siguiente:
 - **iti-271154_uX_nuno_maganda_marco_aurelio_source** (Carpeta con código fuente de la aplicación)
 - **iti-271154_uX_nuno_maganda_marco_aurelio_latex** (Carpeta con código fuente del informe)
 - **iti-271154_uX_nuno_maganda_marco_aurelio.apk** (Instalable (solo si se trata de una aplicación móvil))
 - **iti-271154_uX_nuno_maganda_marco_aurelio.pdf** (Informe)
- Donde:
 - **X** es el número de unidad a un dígito (1, 2, etc)
 - **Sustituir con sus apellidos y nombres de manera apropiada**
- NO DEBE HABER otros archivos .ZIP dentro del ZIP PRINCIPAL

Entregables de proyecto individual (2)

- En el caso que un proyecto individual sea asignado en equipo a varios estudiantes, el archivo entregable DEBE MANEJARSE como la de un proyecto individual
 - Solo un integrante del equipo carga en la plataforma el entregable individual.
 - El informe debe llevar los nombres de los integrantes del equipo que trabajaron (Si se omite a alguien, se asume que no trabajo en el proyecto).
 - NO ES NECESARIO que los otros integrantes marquen en el sistema la tarea como entregada, ya que se conoce su situación desde que se asigna el proyecto. El profesor ya sabe que ustedes van en equipo con el estudiante que hizo la entrega, y por eso deben asegurarse que en el informe entregado, vayan anotados sus nombres.

Entregables de proyectos en equipo

- Crear un archivo ZIP con el siguiente formato de nombre:
 - **iti-271154_eq_NN_uX**
- Dentro, debe contener lo siguiente:
 - **iti-271154_eq_NN_uX_source** (Carpeta con código fuente de la aplicación)
 - **iti-271154_eq_NN_uX_latex** (Carpeta con código fuente del informe)
 - **iti-271154_eq_NN_uX.apk** (Instalable - Solo aplicaciones móviles)
 - **iti-271154_eq_NN_uX.pdf** (Informe)

Donde:

- **NN** es el número de equipo a dos dígitos (01, 02, etc)
 - **X** es el número de unidad a un dígito (1, 2, etc)
- En cada entrega, **UN SOLO INTEGRANTE DEL EQUIPO** deberá cargar los archivos en el classroom.
- NO DEBE HABER otros archivos .ZIP dentro del ZIP PRINCIPAL

Entregables de asignaciones especiales

- Crear un archivo ZIP con el siguiente formato de nombre:
 - **iti-271154_aeX_uY_nuno_maganda_marco_aurelio**
- Dentro, debe contener lo siguiente:
 - **iti-271154_aeX_uY_nuno_maganda_marco_aurelio_source** (Carpeta con código fuente de la aplicación - Cuando aplique)
 - **iti-271154_aeX_uY_nuno_maganda_marco_aurelio_latex** (Carpeta con código fuente del informe o diapositivas)
 - **iti-271154_aeX_uY_nuno_maganda_marco_aurelio.apk** (Instalable - Solo aplicaciones móviles, Cuando aplique)
 - **iti-271154_aeX_uY_nuno_maganda_marco_aurelio.pdf** (Informe)
- Donde:
 - **X** es el número de asignación dentro de la unidad a un dígito (1, 2, etc)
 - **Y** es el número de unidad a un dígito (1, 2, etc)
 - **Sustituir con sus apellidos y nombres de manera apropiada**
- NO DEBE HABER otros archivos .ZIP dentro del ZIP PRINCIPAL

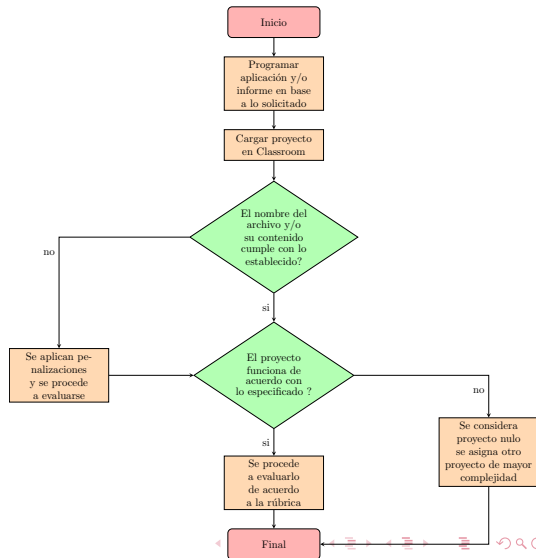
Nombres de Archivos Entregables

En el caso de nombres y apellidos acentuados, con diéresis o con virgulilla (~), sustituir de acuerdo con las siguientes reglas:

- Sustituir N/n por Ñ/ñ
- Sustituir A/a por Á/á
- Sustituir E/e por É/é
- Sustituir I/i por Í/í
- Sustituir O/o por Ó/ó
- Sustituir U/u por Ú/ú
- Sustituir U/u por Ü/ü

Flujo de Evaluación

- Proyecto que funcione pero no siga las indicaciones de entrega, será penalizado.
- Proyecto que no funcione, será anulado
- Se recomienda LEER con cuidado la sección de entregables en esta diapositiva.



Penalizaciones por Entregas Incompletas

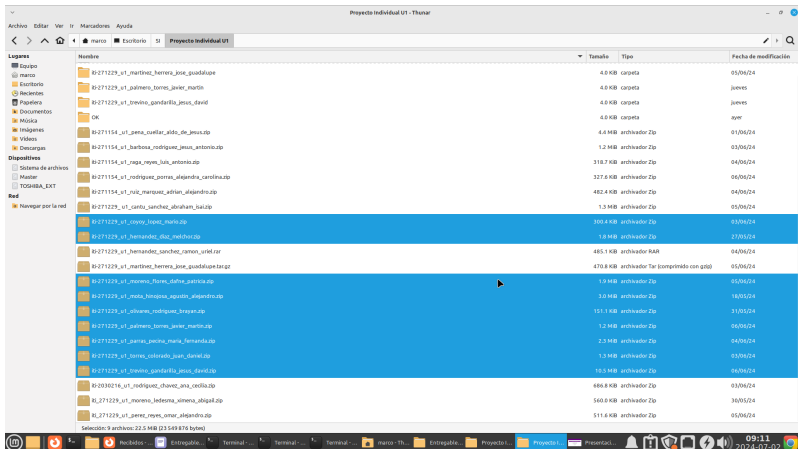
- Proyecto que no este entregado de acuerdo con las especificaciones, será penalizado. Dos escenarios posibles:
 - El proyecto puede revisarse (completo o con faltas al formato).
 - El proyecto NO puede revisarse (falta código fuente, informe, APK, no se compila por alguna falla, etc). En automático el proyecto queda descartado.

Se recomienda LEER con cuidado la sección de entregables de esta presentación. Las penalizaciones son acumulables.

Detalle	Puntos de penalización sobre calificación final
Nombre Archivo	8
Tipo de Archivo	8
Estructura de Directorios	8
Falta o Error en Script	8
Poner ZIPs dentro del ZIP	8
Incluir ejecutables (aplica solo cuando el lenguaje es C++)	8

Premio a la Compresión Lectora 2024

A pesar de las instrucciones en esta presentación, alguien va a hacer las cosas mal



Salon de la Fama de Entregas Completas e Incompletas

Nombre
 iti-271229_u1_trevino_gandarilla_jesus_david_source
 iti-271229_u1_trevino_gandarilla_jesus_david_latex
 iti-271229_u1_trevino_gandarilla_jesus_david.pdf

Nombre
 iti-271229_u1_coyoy_lopez_mario_source
 iti-271229_u1_coyoy_lopez_mario_latex
 iti_271229_u1_coyoy_lopez_mario.pdf

Nombre
 iti-271229_u1_olivares_rodriguez_brayan_source
 iti-271229_u1_olivares_rodriguez_brayan_latex
 iti-271229_u1_olivares_rodriguez_brayan.pdf

Nombre
 iti-271229_u1_martinez_herrera_jose_guadalupe_source
 REPORTE INDIVIDUAL U2.zip
 REPORTE_INDIVIDUAL_U2.pdf

Nombre
 iti-271154_u1_rodriguez_porras_alejandra_carolina_latex.zip
 iti-271154_u1_rodriguez_porras_alejandra_carolina_source.zip
 iti_271154_u1_rodriguez_porras_alejandra_carolina_latex .pdf

Nombre
 iti-271229_u1_parras_pecina_maria_fernanda_latex
 iti-271229_u1_parras_pecina_maria_fernanda_source

Respecto a las Asignaciones Especiales

Importante

Si en algún momento del curso, la asignación especial consiste en desarrollar una presentación de un tema explicado en un BLOG, capítulo de libro, tutorial, etc., el LENGUAJE en el que deben hacerse las DIAPOSITIVAS es el MISMO que en el que está explicado dicho BLOG (a menos que se establezca un lenguaje diferente de manera explícita en el momento de asignar dicha tarea).

Fechas importantes de entrega de proyectos (1)

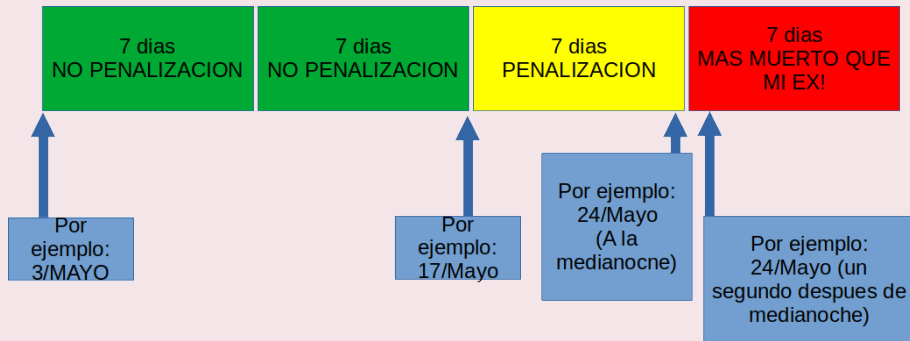
- Fecha de asignación: fecha en que se da a conocer al grupo el trabajo a elaborar
- Fecha de entrega sin penalización: 14 días naturales después de la fecha de asignación
- Proyecto entregado después de la fecha de penalización se le aplica una penalización de 25 PUNTOS
- Fecha de cierre: 21 días naturales después de la fecha de asignación.

Regla "CANTU"

- Ningún proyecto será revisado después de la fecha de cierre. Aún cuando la plataforma permita la entrega tardía, si la última actualización (commit) está fuera de la fecha máxima de entrega (a menos que se haya negociado una extensión con el profesor, y haya evidencia de dicha extensión), dicho proyecto es anulado.

Fechas importantes de entrega de proyectos (2)

Grafo "DAFNE"



- En el momento de publicar la tarea, se incluirá la fecha para no penalización y fecha de cierre.

¿Es posible obtener una calificación negativa?

SI

Calificación asignada después de revisión	70
(-) Debería entregarse 17/Mayo, lo entregó 24/Mayo (1 minuto antes de la medianoche)	25
(-) Nombre Archivo MAL	8
(-) Tipo de Archivo MAL	8
(-) Estructura de Directorios INCORRECTA	8
(-) Faltas o Errores en Script	8
(-) ZIPs dentro del ZIP	8
(-) Incluir ejecutables (aplica solo cuando el lenguaje es C++)	8
(-) Penalización por Fragmentación de Equipo	25
Calificación Final	-28

LINUX

En orden de dificultad

- Linux instalado de manera emulada usando VirtualBox o VMWare.
- Crear una USB o HD booteable (con persistencia) y bootear desde su laptop solo para las clases y los proyectos.
- Linux instalado de manera nativa. Distribuciones recomendadas: **Mint, Ubuntu, Lubuntu, Xubuntu, Debian**

**** Tienen la opción de no INSTALAR LINUX, pero la evaluación será realiza en una PC con Linux instalado**

Software Utilizado

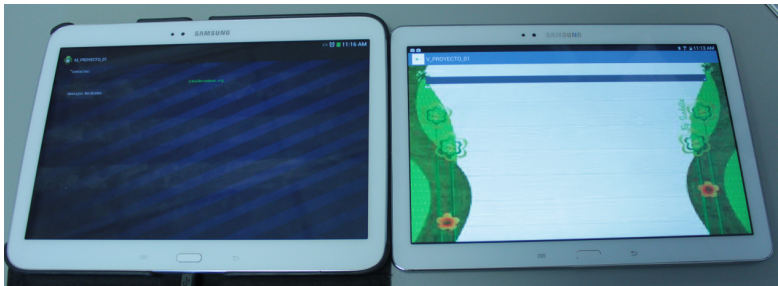
Sobre una instalación de Linux, se debe instalar lo siguiente:

- LaTeX para edición de reportes
- Python3
- PyQt6
- OpenCV
- Numpy, matplotlib, scikit-learn

Se buscan integrantes para ingresar al
Salon de la fama del PLAGIO

Plagio

- Reprobación automática a quien reproduzca códigos de otros compañeros y los reporte como suyos, además de una nota en su expediente con copia para el consejo de calidad



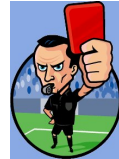
- Reprobación automática a quien copie códigos de Internet y los reporte como suyos, además de una nota en su expediente con copia para el consejo de calidad



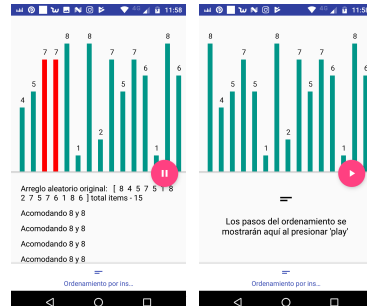
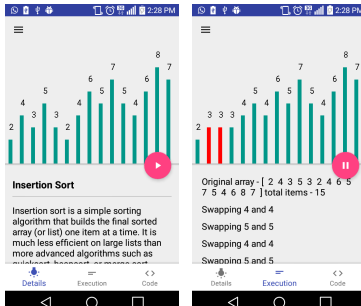
```

1 //Histogram equalization using C++ Image Processing | Programming Techniques - Mustafa Firdous
2
3 #include <iostream>
4 #include <opencv2/imgproc/imgproc.hpp>
5 #include <opencv2/highgui/highgui.hpp>
6 using namespace cv;
7 using namespace std;
8
9 void initHist(Mat image, int histogram[]);
10
11 // initialization all intensity values to 0
12 for(int i = 0; i < 256; i++)
13 {
14     histogram[i] = 0;
15 }
16
17 // calculate the use of pixels for each intensity values
18 for(int i = 0; i < image.rows; i++)
19 {
20     for(int j = 0; j < image.cols; j++)
21     {
22         histogram[image.at<uchar>(i,j)]++;
23     }
24 }
25
26 void combHist(int histogram[], int combHistogram[])
27 {
28     combHistogram[0] = histogram[0];
29     for(int i = 1; i < 256; i++)
30     {
31         combHistogram[i] = combHistogram[i-1] + histogram[i];
32     }
33 }
34
35 //function que devuelve los histogramas
36 void histEqualization(Mat histogram[], const char* name)
37 {
38     int hist[256];
39     for(int i = 0; i < 256; i++)
40     {
41         hist[i] = histogram[i];
42     }
43
44     //calculando los histogramas
45     int hist_w = 128; int hist_h = 128;
46     int bin_w = cvRound( (double) hist_w/256);
47     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
48
49     //devolvendo la maxima intensidad del histograma
50     int max = hist[0];
51     for(int i = 1; i < 256; i++)
52     {
53         if(max < hist[i])
54             max = hist[i];
55     }
56
57     //normalizing the histogram between 0 and 255
58     for(int i = 0; i < 256; i++)
59     {
60         hist[i] = (hist[i] * 255) / max;
61     }
62 }
63
64 //function que devuelve los histogramas
65 void histEqualization(Mat histogram[], const char* name)
66 {
67     int hist[256];
68     for(int i = 0; i < 256; i++)
69     {
70         hist[i] = histogram[i];
71     }
72
73     //calculando los histogramas
74     int hist_w = 128; int hist_h = 128;
75     int bin_w = cvRound( (double) hist_w/256);
76     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
77
78     //devolvendo la maxima intensidad del histograma
79     int max = hist[0];
80     for(int i = 1; i < 256; i++)
81     {
82         if(max < hist[i])
83             max = hist[i];
84     }
85
86     //normalizing the histogram between 0 and 255
87     for(int i = 0; i < 256; i++)
88     {
89         hist[i] = (hist[i] * 255) / max;
90     }
91 }
92
93 //function que devuelve los histogramas
94 void histEqualization(Mat histogram[], const char* name)
95 {
96     int hist[256];
97     for(int i = 0; i < 256; i++)
98     {
99         hist[i] = histogram[i];
100     }
101
102     //calculando los histogramas
103     int hist_w = 128; int hist_h = 128;
104     int bin_w = cvRound( (double) hist_w/256);
105     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
106
107     //devolvendo la maxima intensidad del histograma
108     int max = hist[0];
109     for(int i = 1; i < 256; i++)
110     {
111         if(max < hist[i])
112             max = hist[i];
113     }
114
115     //normalizing the histogram between 0 and 255
116     for(int i = 0; i < 256; i++)
117     {
118         hist[i] = (hist[i] * 255) / max;
119     }
120 }
121
122 //function que devuelve los histogramas
123 void histEqualization(Mat histogram[], const char* name)
124 {
125     int hist[256];
126     for(int i = 0; i < 256; i++)
127     {
128         hist[i] = histogram[i];
129     }
130
131     //calculando los histogramas
132     int hist_w = 128; int hist_h = 128;
133     int bin_w = cvRound( (double) hist_w/256);
134     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
135
136     //devolvendo la maxima intensidad del histograma
137     int max = hist[0];
138     for(int i = 1; i < 256; i++)
139     {
140         if(max < hist[i])
141             max = hist[i];
142     }
143
144     //normalizing the histogram between 0 and 255
145     for(int i = 0; i < 256; i++)
146     {
147         hist[i] = (hist[i] * 255) / max;
148     }
149 }
150
151 //function que devuelve los histogramas
152 void histEqualization(Mat histogram[], const char* name)
153 {
154     int hist[256];
155     for(int i = 0; i < 256; i++)
156     {
157         hist[i] = histogram[i];
158     }
159
160     //calculando los histogramas
161     int hist_w = 128; int hist_h = 128;
162     int bin_w = cvRound( (double) hist_w/256);
163     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
164
165     //devolvendo la maxima intensidad del histograma
166     int max = hist[0];
167     for(int i = 1; i < 256; i++)
168     {
169         if(max < hist[i])
170             max = hist[i];
171     }
172
173     //normalizing the histogram between 0 and 255
174     for(int i = 0; i < 256; i++)
175     {
176         hist[i] = (hist[i] * 255) / max;
177     }
178 }
179
180 //function que devuelve los histogramas
181 void histEqualization(Mat histogram[], const char* name)
182 {
183     int hist[256];
184     for(int i = 0; i < 256; i++)
185     {
186         hist[i] = histogram[i];
187     }
188
189     //calculando los histogramas
190     int hist_w = 128; int hist_h = 128;
191     int bin_w = cvRound( (double) hist_w/256);
192     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
193
194     //devolvendo la maxima intensidad del histograma
195     int max = hist[0];
196     for(int i = 1; i < 256; i++)
197     {
198         if(max < hist[i])
199             max = hist[i];
200     }
201
202     //normalizing the histogram between 0 and 255
203     for(int i = 0; i < 256; i++)
204     {
205         hist[i] = (hist[i] * 255) / max;
206     }
207 }
208
209 //function que devuelve los histogramas
210 void histEqualization(Mat histogram[], const char* name)
211 {
212     int hist[256];
213     for(int i = 0; i < 256; i++)
214     {
215         hist[i] = histogram[i];
216     }
217
218     //calculando los histogramas
219     int hist_w = 128; int hist_h = 128;
220     int bin_w = cvRound( (double) hist_w/256);
221     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
222
223     //devolvendo la maxima intensidad del histograma
224     int max = hist[0];
225     for(int i = 1; i < 256; i++)
226     {
227         if(max < hist[i])
228             max = hist[i];
229     }
230
231     //normalizing the histogram between 0 and 255
232     for(int i = 0; i < 256; i++)
233     {
234         hist[i] = (hist[i] * 255) / max;
235     }
236 }
237
238 //function que devuelve los histogramas
239 void histEqualization(Mat histogram[], const char* name)
240 {
241     int hist[256];
242     for(int i = 0; i < 256; i++)
243     {
244         hist[i] = histogram[i];
245     }
246
247     //calculando los histogramas
248     int hist_w = 128; int hist_h = 128;
249     int bin_w = cvRound( (double) hist_w/256);
250     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
251
252     //devolvendo la maxima intensidad del histograma
253     int max = hist[0];
254     for(int i = 1; i < 256; i++)
255     {
256         if(max < hist[i])
257             max = hist[i];
258     }
259
260     //normalizing the histogram between 0 and 255
261     for(int i = 0; i < 256; i++)
262     {
263         hist[i] = (hist[i] * 255) / max;
264     }
265 }
266
267 //function que devuelve los histogramas
268 void histEqualization(Mat histogram[], const char* name)
269 {
270     int hist[256];
271     for(int i = 0; i < 256; i++)
272     {
273         hist[i] = histogram[i];
274     }
275
276     //calculando los histogramas
277     int hist_w = 128; int hist_h = 128;
278     int bin_w = cvRound( (double) hist_w/256);
279     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
280
281     //devolvendo la maxima intensidad del histograma
282     int max = hist[0];
283     for(int i = 1; i < 256; i++)
284     {
285         if(max < hist[i])
286             max = hist[i];
287     }
288
289     //normalizing the histogram between 0 and 255
290     for(int i = 0; i < 256; i++)
291     {
292         hist[i] = (hist[i] * 255) / max;
293     }
294 }
295
296 //function que devuelve los histogramas
297 void histEqualization(Mat histogram[], const char* name)
298 {
299     int hist[256];
300     for(int i = 0; i < 256; i++)
301     {
302         hist[i] = histogram[i];
303     }
304
305     //calculando los histogramas
306     int hist_w = 128; int hist_h = 128;
307     int bin_w = cvRound( (double) hist_w/256);
308     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
309
310     //devolvendo la maxima intensidad del histograma
311     int max = hist[0];
312     for(int i = 1; i < 256; i++)
313     {
314         if(max < hist[i])
315             max = hist[i];
316     }
317
318     //normalizing the histogram between 0 and 255
319     for(int i = 0; i < 256; i++)
320     {
321         hist[i] = (hist[i] * 255) / max;
322     }
323 }
324
325 //function que devuelve los histogramas
326 void histEqualization(Mat histogram[], const char* name)
327 {
328     int hist[256];
329     for(int i = 0; i < 256; i++)
330     {
331         hist[i] = histogram[i];
332     }
333
334     //calculando los histogramas
335     int hist_w = 128; int hist_h = 128;
336     int bin_w = cvRound( (double) hist_w/256);
337     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
338
339     //devolvendo la maxima intensidad del histograma
340     int max = hist[0];
341     for(int i = 1; i < 256; i++)
342     {
343         if(max < hist[i])
344             max = hist[i];
345     }
346
347     //normalizing the histogram between 0 and 255
348     for(int i = 0; i < 256; i++)
349     {
350         hist[i] = (hist[i] * 255) / max;
351     }
352 }
353
354 //function que devuelve los histogramas
355 void histEqualization(Mat histogram[], const char* name)
356 {
357     int hist[256];
358     for(int i = 0; i < 256; i++)
359     {
360         hist[i] = histogram[i];
361     }
362
363     //calculando los histogramas
364     int hist_w = 128; int hist_h = 128;
365     int bin_w = cvRound( (double) hist_w/256);
366     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
367
368     //devolvendo la maxima intensidad del histograma
369     int max = hist[0];
370     for(int i = 1; i < 256; i++)
371     {
372         if(max < hist[i])
373             max = hist[i];
374     }
375
376     //normalizing the histogram between 0 and 255
377     for(int i = 0; i < 256; i++)
378     {
379         hist[i] = (hist[i] * 255) / max;
380     }
381 }
382
383 //function que devuelve los histogramas
384 void histEqualization(Mat histogram[], const char* name)
385 {
386     int hist[256];
387     for(int i = 0; i < 256; i++)
388     {
389         hist[i] = histogram[i];
390     }
391
392     //calculando los histogramas
393     int hist_w = 128; int hist_h = 128;
394     int bin_w = cvRound( (double) hist_w/256);
395     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
396
397     //devolvendo la maxima intensidad del histograma
398     int max = hist[0];
399     for(int i = 1; i < 256; i++)
400     {
401         if(max < hist[i])
402             max = hist[i];
403     }
404
405     //normalizing the histogram between 0 and 255
406     for(int i = 0; i < 256; i++)
407     {
408         hist[i] = (hist[i] * 255) / max;
409     }
410 }
411
412 //function que devuelve los histogramas
413 void histEqualization(Mat histogram[], const char* name)
414 {
415     int hist[256];
416     for(int i = 0; i < 256; i++)
417     {
418         hist[i] = histogram[i];
419     }
420
421     //calculando los histogramas
422     int hist_w = 128; int hist_h = 128;
423     int bin_w = cvRound( (double) hist_w/256);
424     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
425
426     //devolvendo la maxima intensidad del histograma
427     int max = hist[0];
428     for(int i = 1; i < 256; i++)
429     {
430         if(max < hist[i])
431             max = hist[i];
432     }
433
434     //normalizing the histogram between 0 and 255
435     for(int i = 0; i < 256; i++)
436     {
437         hist[i] = (hist[i] * 255) / max;
438     }
439 }
440
441 //function que devuelve los histogramas
442 void histEqualization(Mat histogram[], const char* name)
443 {
444     int hist[256];
445     for(int i = 0; i < 256; i++)
446     {
447         hist[i] = histogram[i];
448     }
449
450     //calculando los histogramas
451     int hist_w = 128; int hist_h = 128;
452     int bin_w = cvRound( (double) hist_w/256);
453     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
454
455     //devolvendo la maxima intensidad del histograma
456     int max = hist[0];
457     for(int i = 1; i < 256; i++)
458     {
459         if(max < hist[i])
460             max = hist[i];
461     }
462
463     //normalizing the histogram between 0 and 255
464     for(int i = 0; i < 256; i++)
465     {
466         hist[i] = (hist[i] * 255) / max;
467     }
468 }
469
470 //function que devuelve los histogramas
471 void histEqualization(Mat histogram[], const char* name)
472 {
473     int hist[256];
474     for(int i = 0; i < 256; i++)
475     {
476         hist[i] = histogram[i];
477     }
478
479     //calculando los histogramas
480     int hist_w = 128; int hist_h = 128;
481     int bin_w = cvRound( (double) hist_w/256);
482     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
483
484     //devolvendo la maxima intensidad del histograma
485     int max = hist[0];
486     for(int i = 1; i < 256; i++)
487     {
488         if(max < hist[i])
489             max = hist[i];
490     }
491
492     //normalizing the histogram between 0 and 255
493     for(int i = 0; i < 256; i++)
494     {
495         hist[i] = (hist[i] * 255) / max;
496     }
497 }
498
499 //function que devuelve los histogramas
500 void histEqualization(Mat histogram[], const char* name)
501 {
502     int hist[256];
503     for(int i = 0; i < 256; i++)
504     {
505         hist[i] = histogram[i];
506     }
507
508     //calculando los histogramas
509     int hist_w = 128; int hist_h = 128;
510     int bin_w = cvRound( (double) hist_w/256);
511     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
512
513     //devolvendo la maxima intensidad del histograma
514     int max = hist[0];
515     for(int i = 1; i < 256; i++)
516     {
517         if(max < hist[i])
518             max = hist[i];
519     }
520
521     //normalizing the histogram between 0 and 255
522     for(int i = 0; i < 256; i++)
523     {
524         hist[i] = (hist[i] * 255) / max;
525     }
526 }
527
528 //function que devuelve los histogramas
529 void histEqualization(Mat histogram[], const char* name)
530 {
531     int hist[256];
532     for(int i = 0; i < 256; i++)
533     {
534         hist[i] = histogram[i];
535     }
536
537     //calculando los histogramas
538     int hist_w = 128; int hist_h = 128;
539     int bin_w = cvRound( (double) hist_w/256);
540     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
541
542     //devolvendo la maxima intensidad del histograma
543     int max = hist[0];
544     for(int i = 1; i < 256; i++)
545     {
546         if(max < hist[i])
547             max = hist[i];
548     }
549
550     //normalizing the histogram between 0 and 255
551     for(int i = 0; i < 256; i++)
552     {
553         hist[i] = (hist[i] * 255) / max;
554     }
555 }
556
557 //function que devuelve los histogramas
558 void histEqualization(Mat histogram[], const char* name)
559 {
560     int hist[256];
561     for(int i = 0; i < 256; i++)
562     {
563         hist[i] = histogram[i];
564     }
565
566     //calculando los histogramas
567     int hist_w = 128; int hist_h = 128;
568     int bin_w = cvRound( (double) hist_w/256);
569     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
570
571     //devolvendo la maxima intensidad del histograma
572     int max = hist[0];
573     for(int i = 1; i < 256; i++)
574     {
575         if(max < hist[i])
576             max = hist[i];
577     }
578
579     //normalizing the histogram between 0 and 255
580     for(int i = 0; i < 256; i++)
581     {
582         hist[i] = (hist[i] * 255) / max;
583     }
584 }
585
586 //function que devuelve los histogramas
587 void histEqualization(Mat histogram[], const char* name)
588 {
589     int hist[256];
590     for(int i = 0; i < 256; i++)
591     {
592         hist[i] = histogram[i];
593     }
594
595     //calculando los histogramas
596     int hist_w = 128; int hist_h = 128;
597     int bin_w = cvRound( (double) hist_w/256);
598     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
599
600     //devolvendo la maxima intensidad del histograma
601     int max = hist[0];
602     for(int i = 1; i < 256; i++)
603     {
604         if(max < hist[i])
605             max = hist[i];
606     }
607
608     //normalizing the histogram between 0 and 255
609     for(int i = 0; i < 256; i++)
610     {
611         hist[i] = (hist[i] * 255) / max;
612     }
613 }
614
615 //function que devuelve los histogramas
616 void histEqualization(Mat histogram[], const char* name)
617 {
618     int hist[256];
619     for(int i = 0; i < 256; i++)
620     {
621         hist[i] = histogram[i];
622     }
623
624     //calculando los histogramas
625     int hist_w = 128; int hist_h = 128;
626     int bin_w = cvRound( (double) hist_w/256);
627     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
628
629     //devolvendo la maxima intensidad del histograma
630     int max = hist[0];
631     for(int i = 1; i < 256; i++)
632     {
633         if(max < hist[i])
634             max = hist[i];
635     }
636
637     //normalizing the histogram between 0 and 255
638     for(int i = 0; i < 256; i++)
639     {
640         hist[i] = (hist[i] * 255) / max;
641     }
642 }
643
644 //function que devuelve los histogramas
645 void histEqualization(Mat histogram[], const char* name)
646 {
647     int hist[256];
648     for(int i = 0; i < 256; i++)
649     {
650         hist[i] = histogram[i];
651     }
652
653     //calculando los histogramas
654     int hist_w = 128; int hist_h = 128;
655     int bin_w = cvRound( (double) hist_w/256);
656     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
657
658     //devolvendo la maxima intensidad del histograma
659     int max = hist[0];
660     for(int i = 1; i < 256; i++)
661     {
662         if(max < hist[i])
663             max = hist[i];
664     }
665
666     //normalizing the histogram between 0 and 255
667     for(int i = 0; i < 256; i++)
668     {
669         hist[i] = (hist[i] * 255) / max;
670     }
671 }
672
673 //function que devuelve los histogramas
674 void histEqualization(Mat histogram[], const char* name)
675 {
676     int hist[256];
677     for(int i = 0; i < 256; i++)
678     {
679         hist[i] = histogram[i];
680     }
681
682     //calculando los histogramas
683     int hist_w = 128; int hist_h = 128;
684     int bin_w = cvRound( (double) hist_w/256);
685     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
686
687     //devolvendo la maxima intensidad del histograma
688     int max = hist[0];
689     for(int i = 1; i < 256; i++)
690     {
691         if(max < hist[i])
692             max = hist[i];
693     }
694
695     //normalizing the histogram between 0 and 255
696     for(int i = 0; i < 256; i++)
697     {
698         hist[i] = (hist[i] * 255) / max;
699     }
700 }
701
702 //function que devuelve los histogramas
703 void histEqualization(Mat histogram[], const char* name)
704 {
705     int hist[256];
706     for(int i = 0; i < 256; i++)
707     {
708         hist[i] = histogram[i];
709     }
710
711     //calculando los histogramas
712     int hist_w = 128; int hist_h = 128;
713     int bin_w = cvRound( (double) hist_w/256);
714     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
715
716     //devolvendo la maxima intensidad del histograma
717     int max = hist[0];
718     for(int i = 1; i < 256; i++)
719     {
720         if(max < hist[i])
721             max = hist[i];
722     }
723
724     //normalizing the histogram between 0 and 255
725     for(int i = 0; i < 256; i++)
726     {
727         hist[i] = (hist[i] * 255) / max;
728     }
729 }
730
731 //function que devuelve los histogramas
732 void histEqualization(Mat histogram[], const char* name)
733 {
734     int hist[256];
735     for(int i = 0; i < 256; i++)
736     {
737         hist[i] = histogram[i];
738     }
739
740     //calculando los histogramas
741     int hist_w = 128; int hist_h = 128;
742     int bin_w = cvRound( (double) hist_w/256);
743     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
744
745     //devolvendo la maxima intensidad del histograma
746     int max = hist[0];
747     for(int i = 1; i < 256; i++)
748     {
749         if(max < hist[i])
750             max = hist[i];
751     }
752
753     //normalizing the histogram between 0 and 255
754     for(int i = 0; i < 256; i++)
755     {
756         hist[i] = (hist[i] * 255) / max;
757     }
758 }
759
760 //function que devuelve los histogramas
761 void histEqualization(Mat histogram[], const char* name)
762 {
763     int hist[256];
764     for(int i = 0; i < 256; i++)
765     {
766         hist[i] = histogram[i];
767     }
768
769     //calculando los histogramas
770     int hist_w = 128; int hist_h = 128;
771     int bin_w = cvRound( (double) hist_w/256);
772     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
773
774     //devolvendo la maxima intensidad del histograma
775     int max = hist[0];
776     for(int i = 1; i < 256; i++)
777     {
778         if(max < hist[i])
779             max = hist[i];
780     }
781
782     //normalizing the histogram between 0 and 255
783     for(int i = 0; i < 256; i++)
784     {
785         hist[i] = (hist[i] * 255) / max;
786     }
787 }
788
789 //function que devuelve los histogramas
790 void histEqualization(Mat histogram[], const char* name)
791 {
792     int hist[256];
793     for(int i = 0; i < 256; i++)
794     {
795         hist[i] = histogram[i];
796     }
797
798     //calculando los histogramas
799     int hist_w = 128; int hist_h = 128;
800     int bin_w = cvRound( (double) hist_w/256);
801     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
802
803     //devolvendo la maxima intensidad del histograma
804     int max = hist[0];
805     for(int i = 1; i < 256; i++)
806     {
807         if(max < hist[i])
808             max = hist[i];
809     }
810
811     //normalizing the histogram between 0 and 255
812     for(int i = 0; i < 256; i++)
813     {
814         hist[i] = (hist[i] * 255) / max;
815     }
816 }
817
818 //function que devuelve los histogramas
819 void histEqualization(Mat histogram[], const char* name)
820 {
821     int hist[256];
822     for(int i = 0; i < 256; i++)
823     {
824         hist[i] = histogram[i];
825     }
826
827     //calculando los histogramas
828     int hist_w = 128; int hist_h = 128;
829     int bin_w = cvRound( (double) hist_w/256);
830     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
831
832     //devolvendo la maxima intensidad del histograma
833     int max = hist[0];
834     for(int i = 1; i < 256; i++)
835     {
836         if(max < hist[i])
837             max = hist[i];
838     }
839
840     //normalizing the histogram between 0 and 255
841     for(int i = 0; i < 256; i++)
842     {
843         hist[i] = (hist[i] * 255) / max;
844     }
845 }
846
847 //function que devuelve los histogramas
848 void histEqualization(Mat histogram[], const char* name)
849 {
850     int hist[256];
851     for(int i = 0; i < 256; i++)
852     {
853         hist[i] = histogram[i];
854     }
855
856     //calculando los histogramas
857     int hist_w = 128; int hist_h = 128;
858     int bin_w = cvRound( (double) hist_w/256);
859     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
860
861     //devolvendo la maxima intensidad del histograma
862     int max = hist[0];
863     for(int i = 1; i < 256; i++)
864     {
865         if(max < hist[i])
866             max = hist[i];
867     }
868
869     //normalizing the histogram between 0 and 255
870     for(int i = 0; i < 256; i++)
871     {
872         hist[i] = (hist[i] * 255) / max;
873     }
874 }
875
876 //function que devuelve los histogramas
877 void histEqualization(Mat histogram[], const char* name)
878 {
879     int hist[256];
880     for(int i = 0; i < 256; i++)
881     {
882         hist[i] = histogram[i];
883     }
884
885     //calculando los histogramas
886     int hist_w = 128; int hist_h = 128;
887     int bin_w = cvRound( (double) hist_w/256);
888     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
889
890     //devolvendo la maxima intensidad del histograma
891     int max = hist[0];
892     for(int i = 1; i < 256; i++)
893     {
894         if(max < hist[i])
895             max = hist[i];
896     }
897
898     //normalizing the histogram between 0 and 255
899     for(int i = 0; i < 256; i++)
900     {
901         hist[i] = (hist[i] * 255) / max;
902     }
903 }
904
905 //function que devuelve los histogramas
906 void histEqualization(Mat histogram[], const char* name)
907 {
908     int hist[256];
909     for(int i = 0; i < 256; i++)
910     {
911         hist[i] = histogram[i];
912     }
913
914     //calculando los histogramas
915     int hist_w = 128; int hist_h = 128;
916     int bin_w = cvRound( (double) hist_w/256);
917     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
918
919     //devolvendo la maxima intensidad del histograma
920     int max = hist[0];
921     for(int i = 1; i < 256; i++)
922     {
923         if(max < hist[i])
924             max = hist[i];
925     }
926
927     //normalizing the histogram between 0 and 255
928     for(int i = 0; i < 256; i++)
929     {
930         hist[i] = (hist[i] * 255) / max;
931     }
932 }
933
934 //function que devuelve los histogramas
935 void histEqualization(Mat histogram[], const char* name)
936 {
937     int hist[256];
938     for(int i = 0; i < 256; i++)
939     {
940         hist[i] = histogram[i];
941     }
942
943     //calculando los histogramas
944     int hist_w = 128; int hist_h = 128;
945     int bin_w = cvRound( (double) hist_w/256);
946     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
947
948     //devolvendo la maxima intensidad del histograma
949     int max = hist[0];
950     for(int i = 1; i < 256; i++)
951     {
952         if(max < hist[i])
953             max = hist[i];
954     }
955
956     //normalizing the histogram between 0 and 255
957     for(int i = 0; i < 256; i++)
958     {
959         hist[i] = (hist[i] * 255) / max;
960     }
961 }
962
963 //function que devuelve los histogramas
964 void histEqualization(Mat histogram[], const char* name)
965 {
966     int hist[256];
967     for(int i = 0; i < 256; i++)
968     {
969         hist[i] = histogram[i];
970     }
971
972     //calculando los histogramas
973     int hist_w = 128; int hist_h = 128;
974     int bin_w = cvRound( (double) hist_w/256);
975     Mat histImageHist_A, hist_w, CV_MatC, Scalar(0, 0, 0, 0);
976
977     //devolvendo la maxima intensidad del histograma
978     int max = hist[0];
979     for(int i = 1; i < 256; i++)
980     {
981         if(max < hist[i])
982             max = hist[i];
983     }
984
985     //normalizing the histogram between 0 and 255
986     for(int i = 0; i < 256; i++)
987     {
```


- Reprobación automática a quien copie códigos de Internet y los reporte como suyos, además de una nota en su expediente con copia para el consejo de calidad



<https://github.com/naman14/AlgorithmVisualizer-Android>



“Finalmente son jóvenes que están en la preparatoria y que deben de leer su convocatoria con toda claridad, si no cumplen con los requisitos, si no pueden leer una convocatoria que dice tienes que traer número uno esto, número dos esto, número tres esto, no están listos para ser **estudiantes de educación superior**, así lo digo con toda claridad”.

Sara Ladrón de Guevara.

Rectora de la Universidad Veracruzana (2013-2017 y 2017-2021).

CONCLUSIÓN

ODIAME AHORA
POR EXIGIRTE
COMO ESTUDIANTE
PARA QUE NO ME TENGAS QUE
ODIAR DESPUÉS
POR HACERTE
UN PROFESIONAL MEDIOCRE